

LAPORAN PROYEK
SIMULASI SMART GATEWAY LOAD BALANCING TIGA
SERVER BERBASIS DOCKER DENGAN MONITORING
REAL-TIME



Disusun Oleh:

Nama: HAMSAH

Nim: 105841120023

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR
2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga laporan proyek berjudul “Simulasi Smart Gateway Load Balancing Tiga Server Berbasis Docker dengan Monitoring Real-Time” ini dapat diselesaikan dengan baik.

Proyek ini dibuat untuk memahami cara kerja load balancing pada sistem terdistribusi. Sistem yang dibangun terdiri dari satu load balancer dan tiga backend server yang dijalankan menggunakan Docker Compose. Load balancer mendukung dua algoritma distribusi beban, yaitu Round Robin dan Least Connection, serta dilengkapi dashboard monitoring real-time berbasis WebSocket.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk penyempurnaan laporan ini. Semoga laporan ini bermanfaat bagi pembaca, khususnya dalam memahami konsep load balancing secara praktis.

Makassar, Juni 2026

HAMSAH

DAFTAR ISI

KATA PENGANTAR.....	2
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Rumusan Masalah	5
1.3 Tujuan.....	5
1.4 Manfaat	6
1.5 Batasan Masalah.....	6
BAB II TINJAUAN PUSTAKA	7
2.1 Load Balancer	7
2.2 Algoritma Round Robin	7
2.3 Algoritma Least Connection	7
2.4 Docker	7
2.5 Node.js dan Express	7
2.6 WebSocket.....	8
BAB III METODOLOGI.....	9
3.1 Metode Pengembangan	9
3.2 Arsitektur Sistem.....	9
3.3 Alat dan Bahan	10
BAB IV IMPLEMENTASI.....	11
4.1 Struktur Proyek	11
4.2 Backend Server	11
4.3 Smart Gateway Load Balancer.....	11
4.4 Konfigurasi Docker Compose.....	12
4.5 Dashboard Monitoring	12
BAB V HASIL DAN PEMBAHASAN	13
5.1 Cara Menjalankan Aplikasi	13
5.2 Pengujian Round Robin	13
5.3 Pengujian Least Connection.....	13

5.4 Perbandingan Algoritma.....	14
BAB VI PENUTUP	15
6.1 Kesimpulan	15
6.2 Saran.....	15
DAFTAR PUSTAKA	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan layanan digital menuntut sistem aplikasi yang mampu menangani banyak request secara bersamaan. Apabila seluruh request hanya diarahkan ke satu server, maka server tersebut dapat mengalami beban berlebih sehingga respons menjadi lambat atau bahkan gagal melayani pengguna.

Load balancing merupakan teknik untuk membagi beban request ke beberapa server backend. Dengan adanya load balancer, beban kerja dapat disebarkan secara lebih merata sehingga sistem menjadi lebih stabil, responsif, dan mudah dikembangkan.

Pada proyek ini dibuat sebuah simulasi **Smart Gateway Load Balancer** yang mengarahkan request ke tiga backend server. Sistem berjalan menggunakan Docker Compose agar setiap komponen dapat dijalankan secara terpisah namun tetap saling terhubung dalam satu jaringan container.

Project ini menggunakan dua algoritma load balancing, yaitu **Round Robin** dan **Least Connection**. Round Robin membagi request secara bergiliran, sedangkan Least Connection memilih server yang memiliki koneksi aktif paling sedikit. Sistem juga dilengkapi dashboard real-time agar distribusi request dapat diamati secara langsung.

1.2 Rumusan Masalah

Rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana membangun simulasi load balancer dengan tiga backend server menggunakan Docker?
2. Bagaimana menerapkan algoritma Round Robin dan Least Connection pada load balancer?
3. Bagaimana menampilkan distribusi request secara real-time melalui dashboard web?
4. Bagaimana membandingkan pola kerja Round Robin dan Least Connection?

1.3 Tujuan

Tujuan dari proyek ini adalah sebagai berikut:

1. Membuat sistem load balancing sederhana dengan satu gateway dan tiga backend server.
2. Mengimplementasikan algoritma Round Robin dan Least Connection.
3. Menjalankan semua service menggunakan Docker Compose.

4. Menyediakan dashboard monitoring real-time berbasis WebSocket.
5. Melakukan pengujian request untuk melihat distribusi beban ke setiap server.

1.4 Manfaat

Manfaat dari proyek ini adalah sebagai berikut:

1. Membantu mahasiswa memahami konsep load balancing secara praktik.
2. Menjadi contoh implementasi sistem terdistribusi sederhana menggunakan Docker.
3. Memberikan gambaran perbedaan antara algoritma Round Robin dan Least Connection.
4. Menjadi bahan pembelajaran untuk mata kuliah jaringan komputer, cloud computing, atau sistem terdistribusi.

1.5 Batasan Masalah

Batasan masalah dalam proyek ini adalah sebagai berikut:

1. Sistem hanya menggunakan tiga backend server.
2. Algoritma yang digunakan hanya Round Robin dan Least Connection.
3. Backend server bersifat simulasi dan hanya memproses request dengan delay acak.
4. Sistem dijalankan secara lokal menggunakan Docker Compose.
5. Sistem belum mencakup deployment cloud, SSL/TLS, autoscaling, dan failover otomatis.

BAB II

TINJAUAN PUSTAKA

2.1 Load Balancer

Load balancer adalah komponen yang bertugas membagi lalu lintas request ke beberapa server backend. Tujuannya adalah menghindari penumpukan beban pada satu server, meningkatkan ketersediaan layanan, dan menjaga performa sistem.

2.2 Algoritma Round Robin

Round Robin adalah algoritma load balancing yang membagi request secara bergiliran ke setiap server. Apabila terdapat tiga server, maka request pertama dikirim ke server-a, request kedua ke server-b, request ketiga ke server-c, lalu request berikutnya kembali ke server-a.

Kelebihan Round Robin adalah sederhana, ringan, dan mudah diprediksi. Namun, algoritma ini tidak memperhatikan apakah sebuah server sedang sibuk atau tidak.

2.3 Algoritma Least Connection

Least Connection adalah algoritma yang memilih server dengan jumlah koneksi aktif paling sedikit. Algoritma ini lebih adaptif dibandingkan Round Robin karena mempertimbangkan kondisi server saat request masuk.

Algoritma ini cocok digunakan ketika setiap request memiliki waktu proses yang berbeda-beda.

2.4 Docker

Docker adalah platform containerization yang memungkinkan aplikasi dijalankan dalam container. Dengan Docker, setiap service dapat berjalan di lingkungan yang terisolasi dan konsisten.

Docker Compose digunakan untuk menjalankan banyak container sekaligus menggunakan satu file konfigurasi bernama `'docker-compose.yml'`.

2.5 Node.js dan Express

Node.js adalah runtime JavaScript yang dapat digunakan untuk membangun aplikasi server. Express adalah framework web untuk Node.js yang mempermudah pembuatan endpoint API.

Dalam proyek ini, Node.js dan Express digunakan untuk membangun backend server dan load balancer.

2.6 WebSocket

WebSocket adalah protokol komunikasi dua arah yang memungkinkan server mengirim data ke browser secara langsung. Dalam proyek ini, WebSocket digunakan untuk memperbarui dashboard secara real-time tanpa perlu melakukan refresh halaman.

BAB III

METODOLOGI

3.1 Metode Pengembangan

Metode pengembangan dilakukan secara bertahap, yaitu:

1. Merancang arsitektur sistem.
2. Membuat backend server simulasi.
3. Membuat load balancer dengan algoritma Round Robin.
4. Menambahkan algoritma Least Connection.
5. Membuat dashboard monitoring real-time.
6. Membuat konfigurasi Docker Compose.
7. Melakukan pengujian request.
8. Menganalisis hasil distribusi request.

3.2 Arsitektur Sistem

Arsitektur sistem pada proyek ini dirancang menggunakan beberapa container yang saling terhubung melalui jaringan Docker. Setiap container memiliki peran masing-masing dalam proses penerimaan, penerusan, dan pemrosesan request dari pengguna. Komponen utama dalam sistem ini terdiri dari Smart Gateway Load Balancer sebagai penghubung utama dan tiga backend server sebagai penerima distribusi request. Rincian komponen sistem dapat dilihat pada tabel berikut.

No.	Komponen	Port	Fungsi
1	Smart Gateway Load Balancer	3000	Menerima request dari pengguna dan meneruskannya ke backend server
2	Server Akademik	Internal 3000	Backend server pertama
3	Server Administrasi	Internal 3000	Backend server kedua
4	Server Layanan	Internal 3000	Backend server ketiga

Berdasarkan tabel di atas, Smart Gateway Load Balancer berfungsi sebagai pintu masuk utama yang dapat diakses oleh pengguna melalui browser dengan alamat localhost:3000. Setiap request yang masuk akan diterima oleh gateway, kemudian diteruskan ke salah satu backend server yang tersedia.

Tiga backend server yang digunakan adalah Server Akademik, Server Administrasi, dan Server Layanan. Ketiga server tersebut berjalan pada port internal 3000 di dalam jaringan Docker, sehingga tidak dapat diakses langsung dari luar. Dengan adanya pembagian ini, sistem dapat mendistribusikan request ke beberapa server agar beban kerja tidak hanya ditangani oleh satu backend server saja.

3.3 Alat dan Bahan

Alat dan bahan yang digunakan dalam proyek ini terdiri dari beberapa teknologi pendukung untuk membangun, menjalankan, dan menguji sistem load balancing. Setiap teknologi memiliki peran masing-masing, mulai dari pembuatan backend, dashboard, container, hingga pengujian request.

No.	Alat/Teknologi	Keterangan
1	Node.js	Runtime untuk menjalankan backend server dan gateway
2	Express.js	Framework untuk membangun API server
3	WebSocket/ws	Digunakan untuk komunikasi real-time pada dashboard
4	Docker	Menjalankan aplikasi dalam bentuk container
5	Docker Compose	Menjalankan beberapa container sekaligus secara bersamaan
6	HTML/CSS/JavaScript	Digunakan untuk membuat tampilan dashboard
7	Python	Digunakan sebagai script pengujian request dari terminal

Berdasarkan tabel di atas, Node.js dan Express.js digunakan untuk membangun server backend dan gateway. Docker dan Docker Compose digunakan untuk menjalankan seluruh layanan dalam container, sedangkan WebSocket/ws, HTML, CSS, dan JavaScript mendukung tampilan dashboard secara real-time. Python digunakan untuk melakukan pengujian request melalui terminal.

BAB IV

IMPLEMENTASI

4.1 Struktur Proyek

Struktur folder project adalah sebagai berikut:

Text smart-gateway-load-balancer-3server/

```
|— backend/
|— gateway/
|— scripts/
|— hasil-pengujian/
|— laporan/
|— tangkapan-layar/
|— docker-compose.yml
|— README.md
```

4.2 Backend Server

Backend server dibuat menggunakan Node.js dan Express. Setiap server menggunakan kode yang sama, tetapi memiliki identitas berbeda melalui environment variable.

Endpoint	Method	Fungsi
/health	GET	Mengecek status server
/stats	GET	Menampilkan statistik server
/work	GET	Memproses request simulasi dengan delay acak

Setiap request yang masuk ke /work akan menambah koneksi aktif, memproses delay acak, lalu mengurangi koneksi aktif setelah selesai.

4.3 Smart Gateway Load Balancer

Smart Gateway adalah komponen utama yang menerima request dari pengguna. Gateway memilih backend server berdasarkan algoritma yang aktif.

Fungsi utama gateway adalah:

1. Menerima request dari browser atau script pengujian.
2. Memilih server tujuan menggunakan Round Robin atau Least Connection.
3. Meneruskan request ke backend server.
4. Menyimpan statistik request.
5. Menyiarkan perubahan data ke dashboard menggunakan WebSocket.

4.4 Konfigurasi Docker Compose

File `docker-compose.yml` menjalankan empat service, yaitu:

1. server-a
2. server-b
3. server-c
4. gateway

Ketiga backend server hanya menggunakan `expose`, sehingga tidak langsung terbuka ke host. Gateway menggunakan port mapping `3000:3000`, sehingga dapat diakses dari browser.

4.5 Dashboard Monitoring

Dashboard web menampilkan:

1. Status koneksi WebSocket.
2. Algoritma yang sedang digunakan.
3. Status online/offline setiap server.
4. Koneksi aktif setiap server.
5. Total request setiap server.
6. Latency terakhir.
7. Log aktivitas request secara real-time.
8. Tombol pengujian satu request dan batch request.

BAB V

HASIL DAN PEMBAHASAN

5.1 Cara Menjalankan Aplikasi

Langkah menjalankan aplikasi adalah sebagai berikut:

1. Buka folder project di VS Code.
2. Pastikan Docker Desktop sudah berjalan.
3. Jalankan perintah:

Bash docker compose up --build

4. Buka browser dan akses:

Text `http://localhost:3000`

5. Pilih algoritma dan lakukan pengujian melalui dashboard.

5.2 Pengujian Round Robin

Pada pengujian Round Robin, request dibagi secara bergiliran ke tiga server.

Contoh distribusi 9 request adalah sebagai berikut:

Request ke-	Server Tujuan	Keterangan
1	server-a	Request pertama
2	server-b	Bergiliran
3	server-c	Bergiliran
4	server-a	Siklus baru
5	server-b	Bergiliran
6	server-c	Bergiliran
7	server-a	Siklus baru
8	server-b	Bergiliran
9	server-c	Bergiliran

Hasil ini menunjukkan bahwa Round Robin cocok untuk distribusi beban yang sederhana dan seragam.

5.3 Pengujian Least Connection

Pada pengujian Least Connection, load balancer memilih server yang memiliki koneksi aktif paling sedikit. Karena setiap backend memiliki delay acak, distribusi request tidak selalu berurutan.

Contoh pola kerja:

1. Request pertama masuk ke server-a karena semua server masih kosong.
2. Request kedua dapat masuk ke server-b jika server-a masih memproses request.
3. Request ketiga dapat masuk ke server-c jika server-a dan server-b masih aktif.
4. Request berikutnya akan diarahkan ke server yang lebih cepat selesai atau memiliki koneksi aktif paling sedikit.

Least Connection lebih sesuai untuk kondisi ketika durasi request berbeda-beda.

5.4 Perbandingan Algoritma

Pada sistem load balancing, algoritma memiliki peran penting dalam menentukan server tujuan untuk setiap request. Dua algoritma yang dibandingkan pada pengujian ini adalah Round Robin dan Least Connection, karena keduanya memiliki cara distribusi beban yang berbeda.

Aspek	Round Robin	Least Connection
Pola distribusi	Bergiliran tetap	Berdasarkan koneksi aktif
Kompleksitas	Lebih sederhana	Sedikit lebih kompleks
Respons terhadap beban	Tidak memperhatikan beban aktual	Memperhatikan koneksi aktif
Kelebihan	Ringan dan mudah diprediksi	Lebih adaptif terhadap kondisi server
Kekurangan	Kurang optimal jika salah satu server sedang sibuk	Membutuhkan pengecekan jumlah koneksi aktif
Cocok untuk	Request yang seragam	Request dengan durasi proses berbeda

Berdasarkan tabel di atas, Round Robin mendistribusikan request secara bergiliran sehingga lebih sederhana dan mudah diprediksi. Sementara itu, Least Connection mendistribusikan request berdasarkan jumlah koneksi aktif, sehingga lebih adaptif terhadap kondisi server yang sedang berjalan.

BAB VI

PENUTUP

6.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Project Smart Gateway Load Balancer berhasil dibangun menggunakan Docker Compose dengan satu gateway dan tiga backend server.
2. Algoritma Round Robin berhasil membagi request secara bergiliran ke server-a, server-b, dan server-c.
3. Algoritma Least Connection berhasil memilih server berdasarkan jumlah koneksi aktif paling sedikit.
4. Dashboard real-time berhasil menampilkan statistik server, total request, koneksi aktif, dan log aktivitas.
5. Docker Compose mempermudah proses menjalankan seluruh komponen sistem secara bersamaan.

6.2 Saran

Saran pengembangan project ini adalah sebagai berikut:

1. Menambahkan health check otomatis yang lebih lengkap.
2. Menambahkan algoritma Weighted Round Robin.
3. Menambahkan grafik historis menggunakan Chart.js.
4. Melakukan pengujian performa menggunakan k6 atau Apache JMeter.
5. Mengembangkan sistem agar dapat dijalankan pada cloud atau Kubernetes.

DAFTAR PUSTAKA

- [1] A. S. Tanenbaum and M. Van Steen, **Distributed Systems: Principles and Paradigms**, 3rd ed. Pearson, 2017.
- [2] Docker Inc., “Docker Documentation,” 2024.
- [3] OpenJS Foundation, “Node.js Documentation,” 2024.
- [4] Express.js Contributors, “Express.js Guide,” 2024.
- [5] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, 2011.